MLP and K-Means for Gaming Addiction Prediction

1. Introduction

In this part of the project, we explored the application of machine learning methods, specifically a **Multi-Layer Perceptron (MLP)** and **K-Means clustering**, to regression and classification tasks on the Gaming and Mental Health dataset. The primary objective was to design, train, and evaluate these models to predict **addiction score** and **addiction level**, using **Accuracy**, **macro F1-score**, and **Mean Squared Error (MSE)** as evaluation metrics.

The dataset comprises 2000 samples represented by 13 numerical features related to gaming habits and mental health. For the MLP, each sample is treated as a 13-dimensional input vector. K-Means was used as an unsupervised clustering approach adapted for classification through majority voting. Given the overlap between addiction levels and the risk of overfitting, attention was given to hyperparameter tuning and model generalization.

2. Methodology

2.1 Data Pre-Processing

All features were normalized using statistics computed only on the training set. For most experiments, **z-score normalization** was applied using the training mean and standard deviation. For the **Chi-square distance experiment**, min-max normalization was used to keep features non-negative. For hyperparameter tuning, the training set was further split using an **80/20 train-validation split**.

- **MLP:** The 13 numerical features were normalized using z-score normalization based on the training set mean and standard deviation. For classification, labels were converted to **one-hot vectors** for training.
- **K-Means:** The normalized feature vectors were clustered using **distance-based assignments**. Cluster labels were assigned using **majority voting** among the samples associated with each centroid.

2.2 Model Architectures

- **MLP:** Multi-Layer Perceptron was implemented from scratch using **NumPy**, including **forward propagation**, **backpropagation**, **mini-batch training**, and **weight updates**. Classification used one hidden layer with **8 Tanh units**, a **Sigmoid output layer**, and **Cross-Entropy loss**. Regression used a **Linear output layer** with **MSE loss** to predict continuous addiction scores.
- **K-Means:** The iterative clustering algorithm was implemented from scratch with **random centroid initialization**, **distance-based cluster assignment**, and centroid updates based on the mean of assigned samples. The number of clusters and maximum iterations were adjustable hyperparameters.

2.3 Training Strategy

For MLP models, we performed a grid search over **learning rate**, **batch size**, **hidden dimension**, **maximum iterations**, and **hidden activation function**. The final classification configuration was selected using **validation macro F1-score**, while the final configuration for regression was selected using **validation MSE**.
For K-Means, we evaluated multiple **cluster counts**, **distance metrics**, and **random centroid initializations**, selecting the final configuration based on **validation accuracy** and **macro F1-score**.

3. Experiments and Results

3.1 Experimental Process

Initially, we tuned only learning rate and maximum iterations, but the results were unstable: **small learning rates converged slowly**, while **larger values sometimes overfitted**. We then expanded the search to activation functions, hidden dimensions, batch sizes, and maximum iterations for MLP models. For K-Means, we automated the evaluation across cluster counts, distance metrics, and random centroid initializations.

3.2 Hyperparameter Tuning

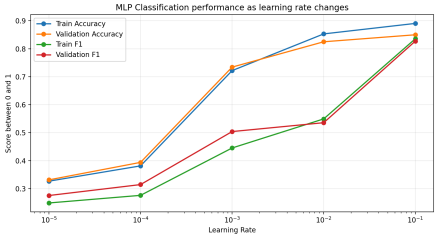
After grid searches stored in **.csv files**, we evaluated **675 MLP classification configurations**, **900 MLP regression configurations**, and **87 K-Means configurations**. Each K-Means configuration was repeated with **5 random centroid initializations**, resulting in 4**35 total K-Means runs**. The best models were selected using **validation macro F1-score** for classification and **validation MSE** for regression.

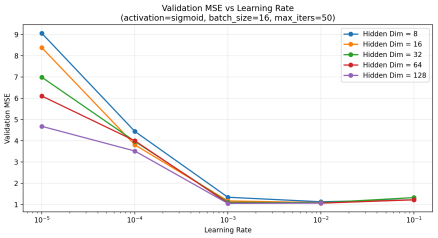
MLP Classification
- Learning rate: 0.1
- Batch size: 32
- Hidden dimensions: 8
- Hidden activation: Tanh
- Output activation: Sigmoid
- Maximum iterations: 200
- Validation accuracy: 85.0%
- Validation F1-score: 0.827

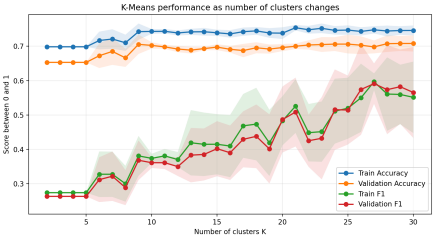
MLP Regression
- Learning rate: 0.001
- Batch size: 16
- Hidden dimensions: 128
- Hidden activation: Sigmoid
- Output activation: Linear
- Maximum iterations: 50
- Validation MSE: 1.048

K-Means
- Number of clusters: 28
- Distance metric: Euclidean
- Maximum iterations: 50
- Validation accuracy: 73.8%
- Validation F1-score: 0.708

For MLP classification, Tanh with a small hidden layer gave the best validation macro F1-score, suggesting that a simple architecture was sufficient. **For MLP regression**, the best validation MSE was obtained with a Sigmoid hidden activation, a large hidden layer, a small learning rate, and a Linear output layer for continuous prediction. This suggests that regression was more sensitive to learning rate and hidden dimension than to longer training.
For K-Means classification, Euclidean distance outperformed Manhattan and Chi-square, so it was selected for the final model.







The validation plots show that larger K improved K-Means performance, while MLP classification performed best with a small hidden layer and learning rate 0.1. For regression, lower learning rate around 0.001 gave the most stable validation MSE.

3.3 Final Model Evaluation

The best models were retrained on the full training set, combining the training and validation splits, and evaluated on the test set. The final results were:
MLP classification: Train Accuracy: **89.06%**, Train F1-score: **0.85**. Test Accuracy: **88.0%**, Test F1-score: **0.84**.
MLP regression: Train MSE: **0.98**. Test MSE: **0.98**.
K-Means classification: Train Accuracy: **73.75%**, Train F1-score: **0.56**. Test Accuracy: **74.25%**, Test F1-score: **0.57**.

3.4 Class Coverage Analysis

MLP Classification on the test set (Ground Truth / Predicted)
Class 0: **280 / 290 samples** Class 1: **107 / 97 samples** Class 2: **13 / 13 samples**

K-Means Classification on the test set (Ground Truth / Predicted)
Class 0: **280 / 370 samples** Class 1: **107 / 12 samples** Class 2: **13 / 18 samples**

The class coverage analysis shows that MLP produced predictions closer to the true class distribution, while K-Means strongly over-predicted **class 0** and under-predicted **class 1**, which explains its lower **macro F1-score**.

4. Discussion and Conclusion

MLP classification clearly outperformed K-Means on the addiction level prediction task. K-Means provided a useful unsupervised baseline, but its lower macro F1-score showed that its accuracy was mainly driven by the majority class and that it struggled with overlapping addiction levels. In contrast, the MLP achieved 88.0% test accuracy and a macro F1-score of 0.84, indicating stronger non-linear class separation and more balanced predictions. For regression, the MLP with a Linear output layer achieved a final test MSE of 0.98, with close train and test errors suggesting good generalization. Overall, **MLP classification was the most effective approach**, while K-Means highlighted the limitations of distance-based clustering on this dataset.